



Publicación de notas:

Lunes 19 de julio de 2021

Revisión:

Miércoles 21 de julio de 2021 de 10:00-12:00h

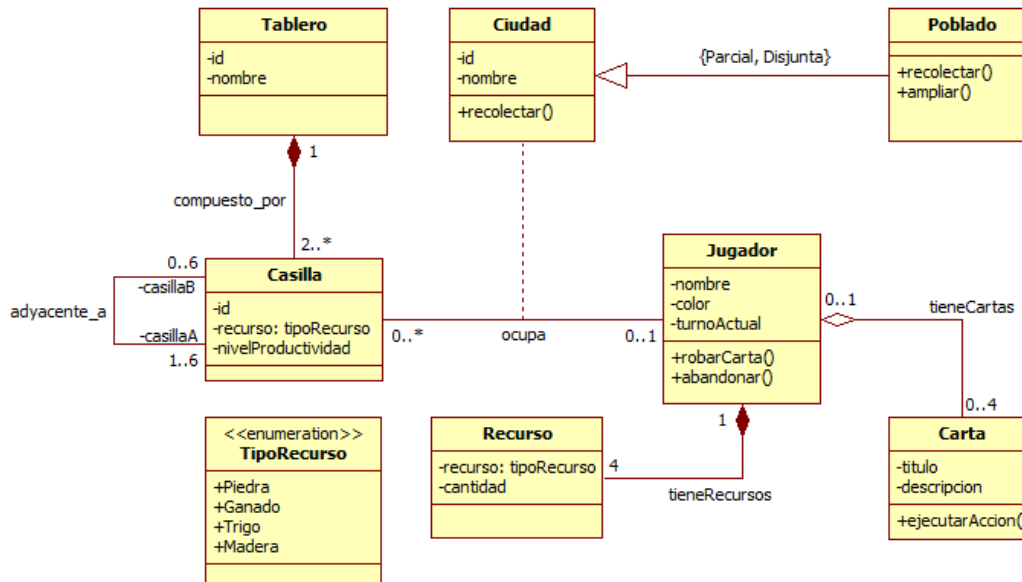
Aula de Examen: _____**Nº asiento ocupado:** _____**DNI con Letra:** _____**Apellidos y Nombre:** _____**FIRMA**

PROBLEMA I (3 puntos)

Una empresa de desarrollo de videojuegos ha iniciado un nuevo proyecto para crear la aplicación software “Colonos de la ETSISI”, una versión digital de un juego de mesa. En este juego hay un tablero, caracterizado por un nombre y un identificador, el cual está compuesto por casillas hexagonales. De cada una de estas casillas se debe conocer su identificador y las casillas adyacentes a ella. Asuma que cada casilla debe estar adyacente al menos a otra, pudiendo estar una casilla completamente rodeada por otras. De cada casilla también se debe conocer el tipo de recurso (Piedra, Ganado, Trigo o Madera) que tiene asociado, así como su nivel de productividad (que se expresa como un número entero entre 1 y 12). Los jugadores pueden ocupar casillas construyendo en ellas dos tipos de edificios diferentes: ciudades y poblados. No es posible ocupar una casilla sin construir un edificio ni construir un edificio sin ocupar una casilla. Por tanto, a raíz de una ocupación siempre se construye un edificio. En ningún caso podrá haber más de un edificio en una misma casilla (es decir, una casilla no podrá ser ocupada por más de un jugador), aunque los jugadores podrán construir múltiples edificios (cada uno en una casilla diferente). De todo edificio se debe conocer su identificador, su nombre, qué jugador lo ha construido y la casilla que ocupa. Los jugadores podrán utilizar durante su turno uno de sus edificios para recolectar unidades del tipo de recurso asociado a la casilla donde éste haya sido construido. La diferencia entre los dos tipos de edificios es que la forma de recolectar recursos de la ciudad es diferente y más eficiente que la del poblado. Además, los poblados, a diferencia de las ciudades, pueden ser ampliados. Siempre que un jugador amplíe un poblado recolectará recursos con dicho poblado. Todo jugador deberá tener un nombre y un color. Además, se deberá conocer en todo momento a qué jugador corresponde el turno actual, así como cuantas unidades de cada tipo de recurso tiene cada jugador. Los jugadores podrán, en determinados momentos, robar cartas hasta tener un máximo de 4. Cada carta tiene un título y un texto que describe la acción que permite ejecutar a los jugadores. La utilización de ciertas cartas requiere a los jugadores robar nuevas cartas. Tenga en cuenta también que, cuando un jugador abandona la partida, sus cartas vuelven a estar disponibles para el resto de jugadores.

El equipo de desarrollo de software de la empresa ha realizado el siguiente **diagrama de clases de análisis** para modelar la aplicación software “Colonos de la ETSISI” descrita anteriormente:

Se asume que todas las clases tienen un método constructor, un método destructor y un método get y otro set para cada uno de sus atributos



Ahora, la jefa de proyecto ha pedido al equipo de desarrollo de software que modifique este diagrama de clases de análisis para incluir los siguientes cambios en el juego:

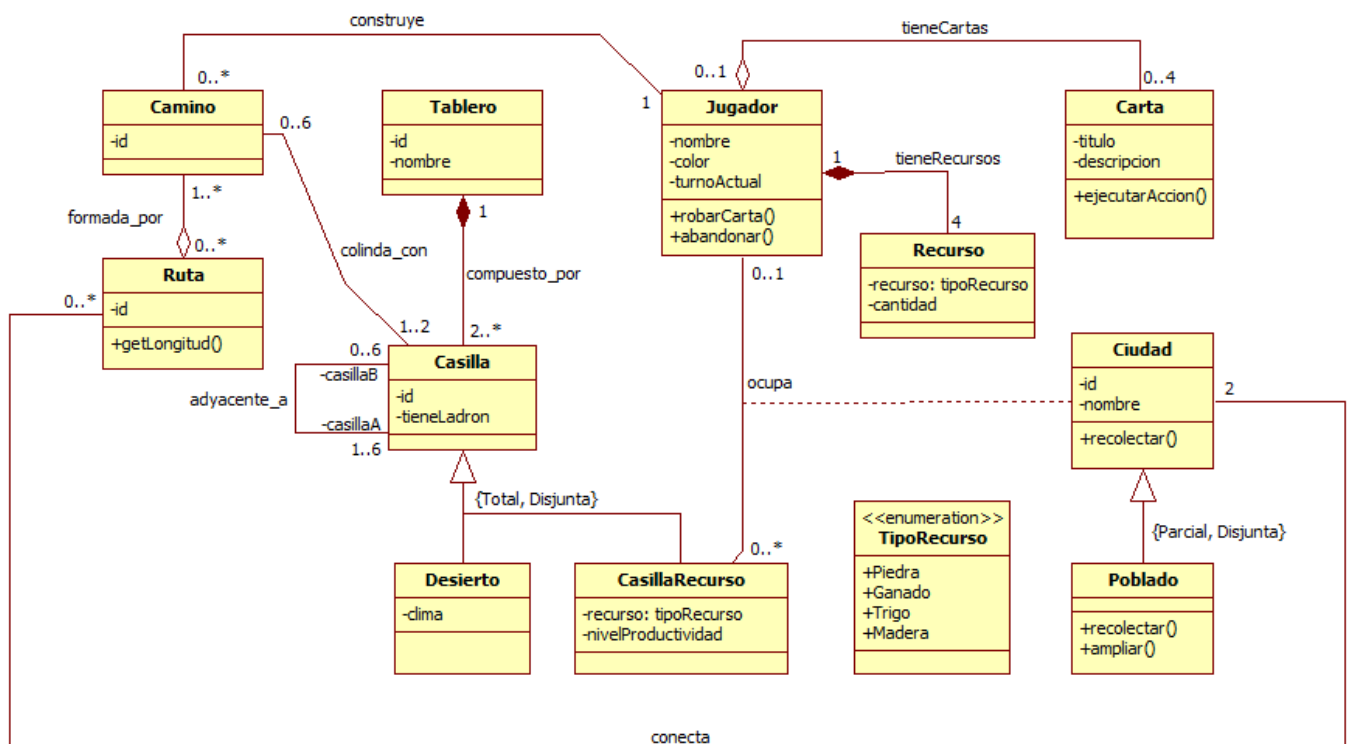
- Hay una casilla especial en el tablero, el ‘Desierto’, la cual no tiene ningún tipo de recurso asociado (tampoco tiene, por tanto, nivel de productividad), y en la cual los jugadores no pueden construir ningún edificio. No obstante, para esta casilla la aplicación deberá conocer en todo momento su clima, ya que éste es relevante para resolver los efectos de algunas cartas.
- En el desierto se encuentra inicialmente el ‘Ladrón’, un personaje que puede ser movido por los jugadores a cualquier casilla del tablero empleando determinadas cartas. La aplicación deberá conocer en todo momento dónde se encuentra el ladrón.
- Los jugadores, además de edificios, pueden construir caminos. Los caminos pueden ser construidos exclusivamente en el borde de las casillas (de cualquier tipo). De todo camino, además de las casillas con las que colinda, se debe saber su identificador y qué jugador lo ha construido.
- La aplicación deberá conocer en todo momento las rutas existentes en el tablero. Una ruta se define como un conjunto de caminos que conecta dos edificios. De cada ruta se deberá conocer su identificador, los caminos que la forman y los edificios que ésta conecta. Tenga en cuenta que un camino puede formar parte de varias rutas y que pueden existir caminos que no formen parte de ninguna ruta. Además de conocer las rutas existentes en el tablero, la aplicación deberá ser capaz de determinar cuál de ellas es la ruta más larga en un momento determinado.

Apartado 1.1 (1,5 puntos)

Modifique y amplíe el **diagrama de clases de análisis** proporcionado en el enunciado para modelar la aplicación software descrita con los cambios solicitados por la jefa de proyecto.

Solución

Se asume que todas las clases tienen un método constructor, un método destructor y un método get y otro set para cada uno de sus atributos



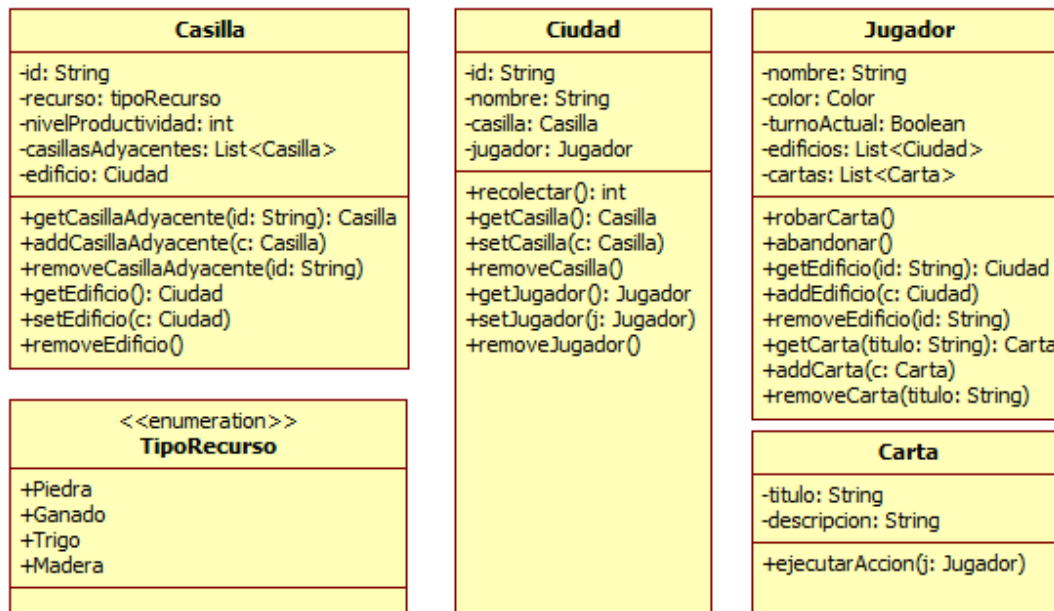
Apartado 1.2 (0,75 puntos)

A partir del **diagrama de clases de análisis** proporcionado en el enunciado y considerando únicamente las clases **Casilla**, **Jugador**, **Carta** y **Ciudad**, construya el **diagrama de clases de diseño refinado para implementación** aplicando únicamente los siguientes refinamientos:

- Añadir tipos de datos a los atributos existentes.
- Eliminar las relaciones convirtiéndolas en atributos de diseño y los métodos necesarios para gestionar dichos atributos adecuadamente.

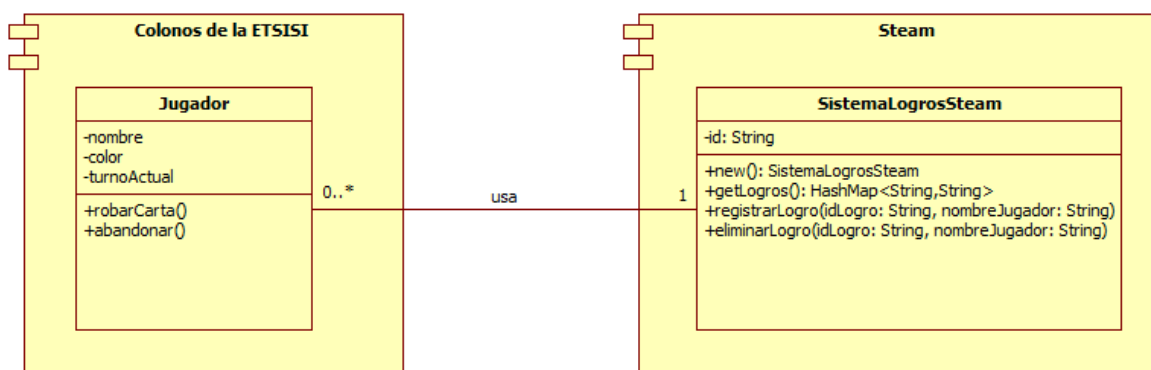
Para la resolución de este apartado, considere que la aplicación se va a implementar en Java, que el método “recolectar” de la clase ‘Ciudad’ devuelve como resultado la cantidad de unidades recolectadas, que el método “ejecutarAccion” de ‘Carta’ recibe como parámetro al jugador que ha usado la carta y que no pueden existir varias cartas con el mismo título.

Solución



Apartado 1.3 (0,75 puntos)

La jefa de proyecto ha solicitado al equipo de desarrollo de software que incorpore una última funcionalidad a la aplicación “Colonos de la ETSISI” a fin de conectarla con *Steam*, una plataforma externa de distribución digital de videojuegos. En concreto, se quiere conectar la aplicación con el sistema de logros de *Steam* para registrar los logros desbloqueados por los jugadores. En la aplicación “Colonos de la ETSISI”, los jugadores pueden desbloquear logros al robar ciertas cartas (y solo de esta manera). Para conectar la aplicación con el sistema de logros de *Steam*, se dispone de un componente de código abierto llamado ‘SistemaLogrosSteam’ facilitado por esta plataforma, el cual permite obtener los logros desbloqueados por un jugador (método *getLogros*), registrar nuevos logros (método *registrarLogro*) y eliminar logros previamente registrados (método *eliminarLogro*). El equipo de desarrollo de software de la empresa ha propuesto el siguiente diseño para implementar esta nueva funcionalidad:



NOTA: Para cada componente se muestran únicamente las clases involucradas en la funcionalidad.

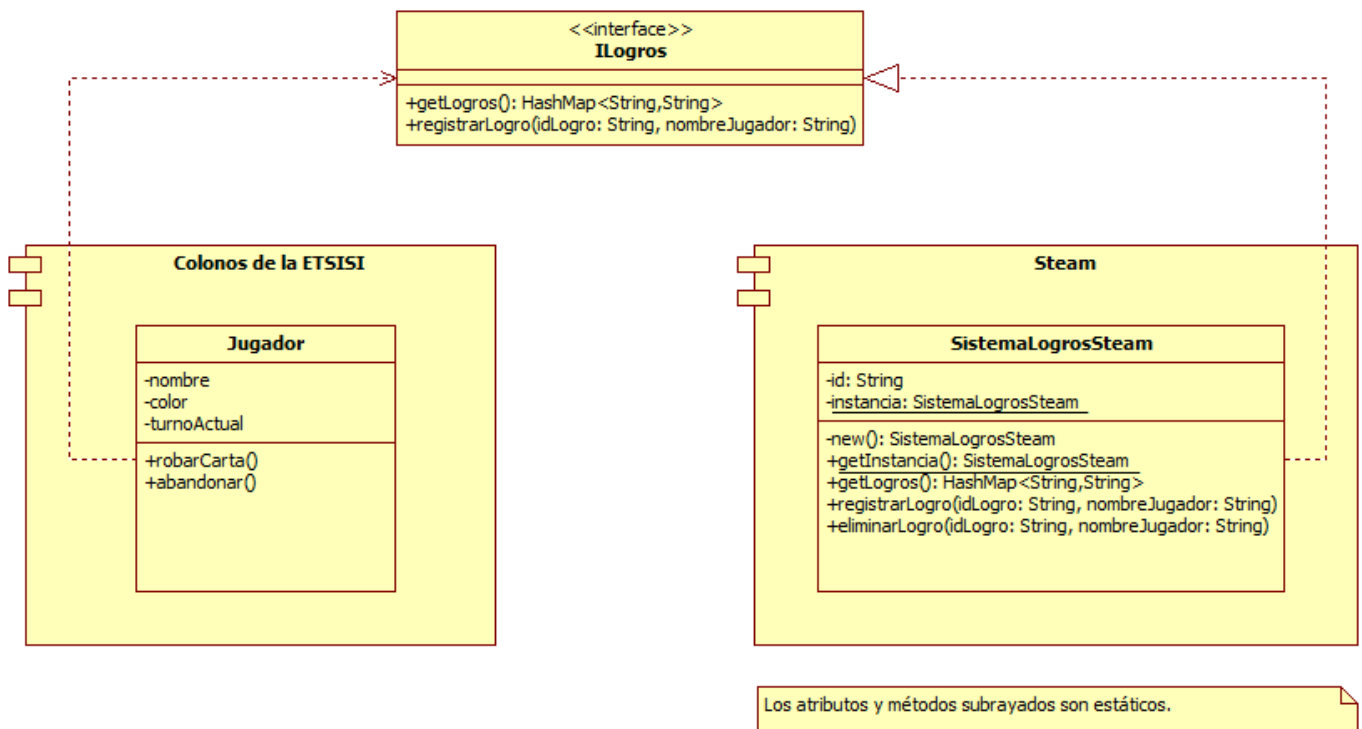
Mejore el diseño propuesto en el enunciado aplicando los principios y patrones de diseño que estime oportunos teniendo en cuenta que:

- Se desea tener el mínimo acoplamiento posible entre los dos componentes representados.
- La aplicación “Colonos de la ETSISI” necesita registrar nuevos logros, así como obtener los logros previamente registrados, pero no eliminar logros.
- Crear un objeto de la clase “SistemaLogrosSteam” es un proceso costoso en tiempo, por lo que se desea garantizar que la aplicación “Colonos de la ETSISI” creará, como máximo, un único objeto de esta clase, con independencia de la cantidad de jugadores.
- Se puede modificar cualquiera de los componentes.

Además de elaborar un diagrama con su propuesta de mejora, **indique los principios y patrones de diseño aplicados**.

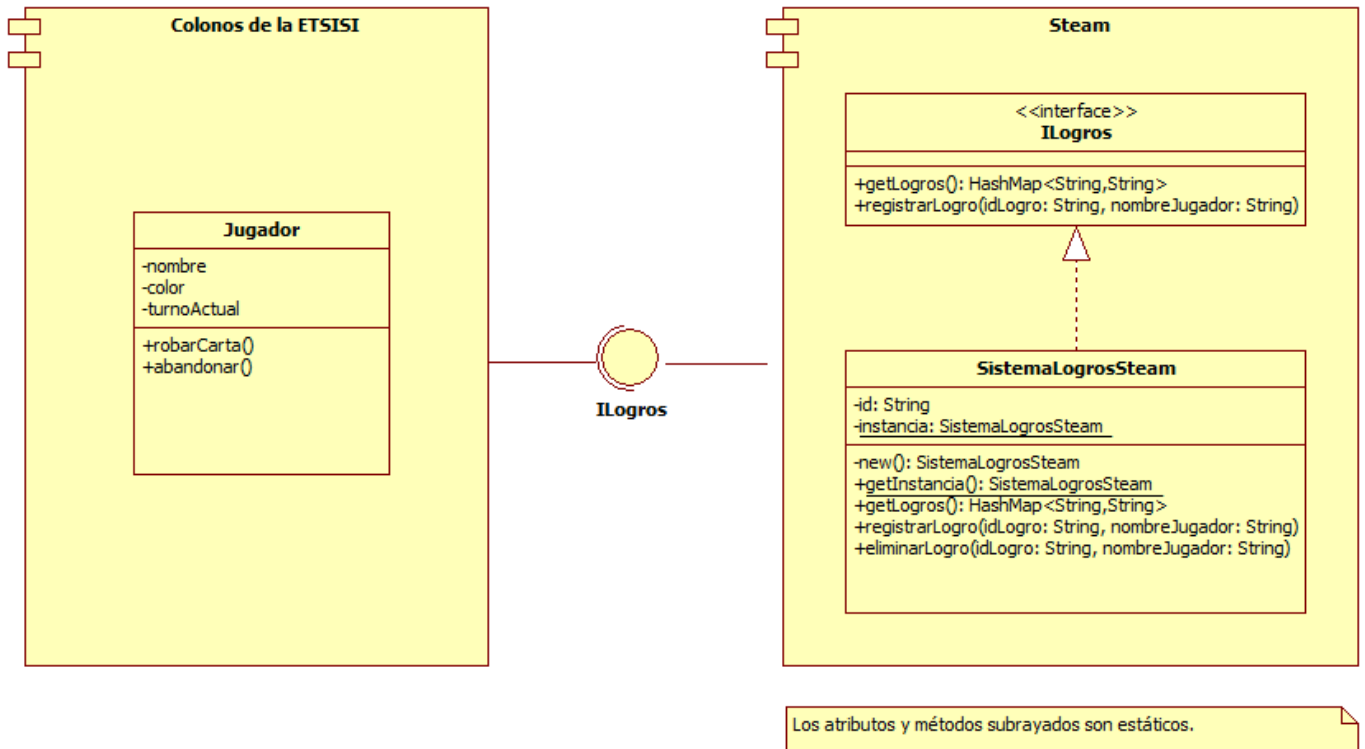
Solución

Opción A



Se ha aplicado el principio SOLID de inversión de dependencias y el patrón de diseño Singleton.

Opción B



Se ha aplicado el principio SOLID de inversión de dependencias y el patrón de diseño Singleton.